

1. Diagrama de Arquitectura General

Este diagrama muestra los componentes principales del backend y cómo interactúan entre sí.

graph TD

subgraph Cliente/Frontend

A[Navegador/App Móvil] --> B(API Gateway / Load Balancer)

end

subgraph Red Docker Compose

B --> C[ProductsService]

B --> D[InventoryService]

C --> E[SQL Server (ProductsDb)]

D --> F[SQL Server (InventoryDb)]

D --> C

end

style A fill:#f9f,stroke:#333,stroke-width:2px

style B fill:#bbf,stroke:#333,stroke-width:2px

style C fill:#ccf,stroke:#333,stroke-width:2px

style D fill:#ccf,stroke:#333,stroke-width:2px

style E fill:#fcc,stroke:#333,stroke-width:2px

style F fill:#fcc,stroke:#333,stroke-width:2px

linkStyle 0 stroke:#00f,stroke-width:2px,fill:none,stroke-dasharray: 5 5;

linkStyle 1 stroke:#00f,stroke-width:2px,fill:none,stroke-dasharray: 5 5;

linkStyle 2 stroke:#00f,stroke-width:2px,fill:none,stroke-dasharray: 5 5;

linkStyle 3 stroke:#00f,stroke-width:2px,fill:none,stroke-dasharray: 5 5;

linkStyle 4 stroke:#00f,stroke-width:2px,fill:none,stroke-dasharray: 5 5;

linkStyle 5 stroke:#00f,stroke-width:2px,fill:none,stroke-dasharray: 5 5;

subgraph Leyenda

L1[Caja Blanca: Contenedor/Servicio]

L2[Caja Roja: Base de Datos]

L3[Flecha Azul Discontinua: Llamada HTTP/API]

L4[Flecha Negra Sólida: Conexión Directa a DB]

end

Explicación:

- **Cliente/Frontend:** La aplicación que interactúa con el backend.
- **API Gateway / Load Balancer:** Un componente opcional (no implementado explícitamente en este Docker Compose, pero implícito en el mapeo de puertos) que enrutaría las solicitudes al servicio correcto y podría manejar autenticación, limitación de tasa, etc.
- **ProductsService:** Microservicio encargado de la gestión de productos. Se comunica directamente con su propia base de datos **ProductsDb**.
- **InventoryService:** Microservicio encargado de la gestión del inventario. Se comunica directamente con su propia base de datos **InventoryDb**.
- **SQL Server (ProductsDb / InventoryDb):** Instancias de SQL Server corriendo en un contenedor Docker, con bases de datos separadas para cada servicio.
- **Flujo de Comunicación:**
 - El cliente se comunica con los servicios a través de HTTP.
 - Los servicios se comunican con sus bases de datos directamente.
 - **¡Importante!** El **InventoryService** se comunica con el **ProductsService** a través de **llamadas HTTP (ProductApiClient)**, no accediendo directamente a **ProductsDb**.

2. Diagrama de Flujo de Datos (Ejemplo: Crear Producto y Actualizar Inventario)

Este diagrama ilustra un escenario común de interacción entre los servicios.

sequenceDiagram

participant C as Cliente/Frontend

participant PS as ProductsService

participant PDB as ProductsDb

participant IS as InventoryService

participant IDB as InventoryDb

C->>PS: 1. POST /products (Crear Producto)

PS->>PDB: 2. Guarda Producto (INSERT)

PDB-->>PS: 3. Producto Guardado (ID Generado)

PS-->>C: 4. 201 Created (Producto con ID)

Note over C,IDB: Cliente copia el ID del producto

C->>IS: 5. GET /inventories/{productId} (Consultar Inventario)

IS->>PS: 6. HTTP GET /products/{productId} (Verificar Producto)

PS->>PDB: 7. Consulta Producto

PDB-->>PS: 8. Detalles del Producto

PS-->>IS: 9. 200 OK (Detalles del Producto)

IS->>IDB: 10. Consulta Inventario por ProductId

alt Si no existe en InventoryDb

 IDB-->>IS: 11. No encontrado

 IS->>IDB: 12. Inicializa InventoryItem (Quantity: 0)

 IDB-->>IS: 13. InventoryItem Creado

end

IS-->>C: 14. 200 OK (Inventario actual)

C->>IS: 15. PATCH /inventories/{productId} (Actualizar Cantidad)

IS->>PS: 16. HTTP GET /products/{productId} (Verificar Producto)

PS->>PDB: 17. Consulta Producto

PDB->>PS: 18. Detalles del Producto

PS->>IS: 19. 200 OK (Detalles del Producto)

IS->>IDB: 20. Consulta InventoryItem por ProductId

IDB->>IS: 21. InventoryItem Existente

IS->>IDB: 22. Actualiza Quantity y LastUpdated

IDB->>IS: 23. Cambios Guardados

IS->>C: 24. 200 OK (Inventario Actualizado)

Explicación del Flujo:

1. **Crear Producto:** El cliente envía una solicitud al **ProductsService**. Este servicio guarda el producto en su **ProductsDb** y devuelve el ID generado al cliente.
2. **Consultar Inventario:** El cliente solicita el inventario de un **productId** al **InventoryService**.
3. **Verificación Inter-Servicio:** El **InventoryService** **no accede directamente a ProductsDb**. En su lugar, utiliza su **ProductApiClient** para hacer una llamada HTTP al **ProductsService** para verificar la existencia del producto y obtener sus detalles.
4. **Lógica de Inventario:** Una vez verificado el producto, el **InventoryService** consulta su propia **InventoryDb**. Si el **InventoryItem** para ese **productId** no existe, lo inicializa con cantidad 0.
5. **Actualizar Inventario:** Cuando el cliente solicita un cambio en la cantidad, el **InventoryService** realiza una verificación similar con **ProductsService** y luego actualiza la cantidad en su **InventoryDb**.

Este flujo demuestra la independencia de los servicios y cómo se comunican a través de APIs bien definidas, manteniendo el aislamiento de sus datos.